

多次元累積和の一般化とその計算効率に関する研究
Generalization of Multi-Dimensional Cumulative Sums
and Their Computational Efficiency

千田研究室 後藤柊矢
Chida Laboratory Shuya GOTO

令和 6 年 xx 月 xx 日

Abstract

This study focuses on the formalization of the theory of multidimensional cumulative sums and the development of efficient computation methods utilizing this theory. While cumulative sums are widely used to quickly compute range sums in one- and two-dimensional data arrays, research on extending this approach to higher-dimensional data and improving its computational efficiency remains insufficient. Building upon existing theories for one- and two-dimensional cumulative sums, this study proposes a generalized method that can be extended to arbitrary dimensions D .

The proposed method constructs an algorithm that achieves a computational complexity of $O(N)$ for preprocessing cumulative sums with data size N and $O(2^D)$ for query processing. By applying the principle of inclusion-exclusion, the method efficiently computes the total sum over arbitrary subregions in a multidimensional space. Additionally, leveraging bit representations for indices optimizes set operations in higher dimensions, simplifying the implementation process.

The findings of this study provide valuable solutions to both academic and practical challenges aimed at the efficient analysis of high-dimensional data. In particular, the generalization of multidimensional cumulative sums and insights into improving their computational efficiency are expected to contribute significantly to the advancement of high-dimensional data processing techniques.

目次

第 1 章	序論	3
1.1	背景	3
1.2	目的	3
第 2 章	準備	5
2.1	集合論	5
2.1.1	記法	5
2.1.2	集合の演算	5
2.1.3	包除原理	6
2.1.4	重み付き包除原理	7
2.2	計算量	7
2.2.1	O 記法	7
2.2.2	O 記法の性質	8
2.3	群	8
2.3.1	群の定義	8
2.3.2	可換群	8
2.3.3	群の性質	8
第 3 章	既存手法	10
3.1	一次元累積和	10
3.1.1	定義	10
3.1.2	構築	10
3.1.3	クエリ処理	10
3.2	二次元累積和	11
3.2.1	定義	11
3.2.2	構築	11
3.2.3	クエリ処理	12
第 4 章	提案手法	14
4.1	重み付き包除原理による考察	14
4.2	多次元累積和	15

4.2.1	定義	15
4.2.2	構築	15
4.2.3	クエリ処理	17
第 5 章	今後の展望	20
参考文献		22

第1章

序論

1.1 背景

累積和のアルゴリズムは、データ列に対する区間和の計算を高速に行うための手法として、情報処理のさまざまな場面で広く利用されてきた。一次元累積和の基本的な考え方は、与えられたデータ列に対してあらかじめ前計算をしておくことで、後続のクエリに対して効率的な応答を可能にするというものである。これにより、区間和の計算を単純な反復処理による計算量 $O(N)$ から定数時間の計算量 $O(1)$ に削減することができる。累積和の概念は二次元へと拡張され、文字列処理、数値計算、画像処理などのより広範な分野においても応用が進んだ [1]。

しかしながら、現行の累積和アルゴリズムは主に一次元または二次元のデータを対象としており、人間が視覚的に直感しやすい次元に留まっているのが現状である。三次元以上の高次元データに関する累積和アルゴリズムは十分に一般化されておらず、理論的な整備や実装の試みは限られている。

近年、データサイエンスや機械学習、ビッグデータ解析の発展に伴い、膨大な高次元データを扱う機会が増加している。例えば、画像解析では通常は二次元データが用いられるが、時間軸を加えることで三次元、さらに複数のセンサ情報を統合すれば四次元以上のデータが生成される。同様に、地理情報システム (GIS) や物理シミュレーションでは、空間的次元に時間的要素や他の物理的変数を組み合わせた多次元データが扱われることが一般的である。これらの応用分野において、任意の次元に対応する累積和アルゴリズムの理論的基盤が整備されていれば、これまで計算上の困難が伴っていた高次元データ解析の課題に対して実用的な解決策を提供できる可能性がある。

本研究では、一次元および二次元累積和の基礎理論を出発点として、それを三次元以上の高次元データに拡張するための理論の定式化と計算アルゴリズムの構築を目指した。このような理論の整備は、将来的に高次元データ解析が必要となる際に大きな利便性をもたらすだろう。

1.2 目的

本研究の目的を以下に示す。

1. ビット表記を用いた多次元累積和のアルゴリズムを示し、その正当性を包除原理を用いて

証明する:

高次元データの各次元の添字が複数の集合を定義し、これらの集合に対して包除原理を適用することで任意の部分領域における総和を効率的に計算する仕組みを構築する。この定式化において、添字をビット表記で表現することで、高次元空間における集合演算を効率化し、アルゴリズムの単純化および計算効率の向上を図る。

2. 多次元累積和アルゴリズムのプログラムによる実装を示す:

この実装では、理論で示したアルゴリズムの構築手順を忠実に再現するとともに、プログラムの汎用性および可読性を考慮した設計を行う。

第2章

準備

2.1 集合論

本章では、本研究で用いる集合論の基本的な概念と記法について説明する [2].

2.1.1 記法

- 集合 A の要素数を $|A|$ で表す.
- x が集合 A の元であることを $x \in A$ で表す.

2.1.2 集合の演算

定義 1 (和集合). 二つの集合 A, B について, A と B のいずれか少なくとも一方に含まれるような元の集合を A と B の和集合といい, $A \cup B$ で表す. すなわち,

$$A \cup B := \{x | x \in A \text{ または } x \in B\} \quad (2.1)$$

である.

定義 2 (積集合). 二つの集合 A と B 両方に含まれる元全体の集合を A と B の積集合といい, $A \cap B$ で表す. すなわち,

$$A \cap B := \{x | x \in A \text{ かつ } x \in B\} \quad (2.2)$$

である.

また, 以下に示す結合法則と分配法則が成り立つことが知られている.

定理 1 (結合法則). 集合 A, B, C について以下の式が成り立つ.

$$(A \cup B) \cup C = A \cup (B \cup C) \quad (2.3)$$

$$(A \cap B) \cap C = A \cap (B \cap C) \quad (2.4)$$

定理 2 (分配法則). 集合 A, B, C について以下の式が成り立つ.

$$A \cup (B \cap C) = (A \cup B) \cap (A \cup C) \quad (2.5)$$

$$A \cap (B \cup C) = (A \cap B) \cup (A \cap C) \quad (2.6)$$

2.1.3 包除原理

有限集合 A と B に対して和集合の要素数 $|A \cup B|$ を求めるには, まずそれぞれに属する要素数 $|A|$ と $|B|$ を足し合わせ, その後 A と B の積集合 $|A \cap B|$ の要素数を引けば良い. 具体的には, 以下の式が成り立つ.

$$|A \cup B| = |A| + |B| - |A \cap B| \quad (2.7)$$

また, n 個の集合 $A_1, A_2, \dots, A_n$ の和集合の要素数について, 以下の式が成り立つ.

定理 3 (包除原理).

$$\begin{aligned} \left| \bigcup_{i=1}^n A_i \right| &= \sum_i |A_i| \\ &\quad - \sum_{i < j} |A_i \cap A_j| \\ &\quad + \sum_{i < j < k} |A_i \cap A_j \cap A_k| \\ &\quad - \dots \\ &\quad + (-1)^{n-1} |A_1 \cap \dots \cap A_n|. \end{aligned} \quad (2.8)$$

これを包除原理 (Principle of Inclusion-Exclusion, PIE) という [3].

証明. 集合ちょうど m 個の共通部分は, 右辺第 k 項において $(-1)^{k-1} \binom{m}{k}$ だけ足されるので, 合計で

$$\sum_{k=1}^m (-1)^{k-1} \binom{m}{k}$$

となる.

ここで, 二項定理より

$$\begin{aligned} 0 &= (1 - 1)^m \\ &= \sum_{k=0}^m \binom{m}{k} (-1)^k \\ &= \binom{m}{0} + \sum_{k=1}^m \binom{m}{k} (-1)^k \\ &= 1 - \sum_{k=1}^m \binom{m}{k} (-1)^{k-1} \\ &\therefore \sum_{k=1}^m (-1)^{k-1} \binom{m}{k} = 1 \end{aligned}$$

したがって、どんな m に対してもこの数え上げが常に 1 であることが分かり、包除原理が成り立つことが示された。□

2.1.4 重み付き包除原理

包除原理を拡張し、和集合の各要素に対する重みを考慮した重み付き包除原理 (Weighted Inclusion-Exclusion Principle) を導入する。

有限集合 S_1, S_2 に対して、各要素 i に重み W_i が付与されている場合、和集合の重み総和は以下のように求められる：

$$\sum_{i \in |S_1 \cup S_2|} W_i = \sum_{i \in S_1} W_i + \sum_{i \in S_2} W_i - \sum_{i \in S_1 \cap S_2} W_i \quad (2.9)$$

これを n 個の集合 $S_1, S_2, \dots, S_n$ に拡張すると、次の重み付き包除原理が得られる。

定理 4 (重み付き包除原理). 任意の n 個の集合 $S_1, S_2, \dots, S_n$ に対して、各要素 i に重み W_i がある場合、和集合の重み総和について以下の式が成り立つ：

$$\begin{aligned} \sum_{i \in |\bigcup_{j=1}^n S_j|} W_i &= \sum_j \sum_{i \in S_j} W_i \\ &\quad - \sum_{j_1 < j_2} \sum_{i \in S_{j_1} \cap S_{j_2}} W_i \\ &\quad + \sum_{j_1 < j_2 < j_3} \sum_{i \in S_{j_1} \cap S_{j_2} \cap S_{j_3}} W_i \\ &\quad - \dots \\ &\quad + (-1)^{n-1} \sum_{i \in S_1 \cap \dots \cap S_n} W_i \end{aligned} \quad (2.10)$$

2.2 計算量

本章では、アルゴリズムの効率性を評価する指標として広く用いられる O 記法とその基本的な性質について説明する [2].

2.2.1 O 記法

定義 3 (O 記法). 整数 N に対して定義されている関数 f, g に対し、

$$f(N) = O(g(N))$$

であるとは、ある正の実数 c と 0 以上の整数 N_0 が存在して、任意の $N \geq N_0$ に対して

$$|f(N)| \leq c|g(N)|$$

が成り立つことをいう。

多変数についても同様に定義される。

2.2.2 O 記法の性質

以下に、 O 記法の基本的な性質を示す。

1. 和:

$$O(f(n)) + O(f(n)) = O(f(n)).$$

2. 積:

$$O(f(n))O(g(n)) = O(f(n)g(n)), \quad f(n)O(g(n)) = O(f(n)g(n)).$$

3. 定数倍:

$$O(cf(n)) = O(f(n)) \quad (c \text{ は } 0 \text{ でない定数}).$$

2.3 群

本章では、本研究で用いる代数的構造である群の定義と性質について説明する [4].

2.3.1 群の定義

集合 G とその上の二項演算 $+: G \times G \rightarrow G$ が群であるとは、以下の三つの条件を満たすことである。

1. **結合則:** 任意の $a, b, c \in G$ に対して、 $(a + b) + c = a + (b + c)$ が成り立つ。
2. **単位元の存在:** ある $e \in G$ が存在して、任意の $a \in G$ に対して、 $a + e = e + a = a$ が成り立つ。
3. **逆元の存在:** 任意の $a \in G$ に対して、ある $b \in G$ が存在して、 $a + b = b + a = e$ が成り立つ。

なお本論文では、計算機上での二項演算は $O(1)$ を仮定し、演算結果のことを和と呼ぶこととする。

2.3.2 可換群

群 $(G, +)$ が可換群であるとは、任意の $a, b \in G$ に対して、 $a + b = b + a$ が成り立つことである。

2.3.3 群の性質

本論文で用いる群の性質を以下に示す。

補題 1. 群 $(G, +)$ において、 $-(a + b) = (-b) + (-a)$ が成り立つ。

証明. $a + b + (-b) + (-a) = a + e + (-a) = a + (-a) = e$ より, $(-b) + (-a)$ は $a + b$ の逆元である. $\square$

補題 2. $(G, +)$ を群, $N \in \mathbb{N}$, 1 以上 N 以下の整数 i に対して $A_i \in G$ とする. このとき,

$$-(A_1 + A_2 + \dots + A_N) = -A_N - A_{N-1} - \dots - A_1 \quad (2.11)$$

が成り立つ.

証明. 帰納法を用いて証明する.

$N = 1$ のときは明らかに成り立つ.

$N = k$ のときに

$$-(A_1 + A_2 + \dots + A_k) = -A_k - A_{k-1} - \dots - A_1$$

が成り立つと仮定すると, $N = k + 1$ のとき

$$\begin{aligned} -(A_1 + A_2 + \dots + A_{k+1}) &= -((A_1 + A_2 + \dots + A_k) + A_{k+1}) \\ &= -A_{k+1} - (A_1 + A_2 + \dots + A_k) \quad (\because \text{補題 1 より}) \\ &= -A_{k+1} - A_k - A_{k-1} - \dots - A_1 \quad (\because \text{仮定より}) \end{aligned} \quad (2.12)$$

となる. したがって, $N = k + 1$ のときも成り立つ.

$\square$

第 3 章

既存手法

3.1 一次元累積和

一次元累積和は、与えられた数列に対してその区間和を前計算することで、任意の区間の和を高速に求めるための手法である [5]。具体的には、長さ $N \in \mathbb{N}$ の数列 A に対してクエリ l, r が与えられたとき

$$\sum_{x=l}^r A_x \quad (3.1)$$

の値を高速に求めることができる。

3.1.1 定義

$(G, +)$ を群, 0 をその単位元とし、各要素が G の元である長さ N の数列 $A = \{A_1, A_2, \dots, A_N\}$ が与えられたとき、 A の累積和 $S = \{S_0, S_1, S_2, \dots, S_N\}$ は以下のように定義される。

$$S_0 = 0, \quad S_i = \sum_{x=1}^i A_x \quad (1 \leq i \leq N) \quad (3.2)$$

3.1.2 構築

累積和の構築は、漸化式 $S_i = S_{i-1} + A_i$ を用いて $O(N)$ で行うことができる。

3.1.3 クエリ処理

数列 A の区間 $[l, r]$ の和を求めるクエリ処理は、以下の定理 4 により $O(1)$ で行うことができる。

定理 5. 数列 A の区間 $[l, r]$ の和は、 $-S_{l-1} + S_r$ で求めることができる。

証明.

$$\begin{aligned} -S_{l-1} + S_r &= -\sum_{x=1}^{l-1} A_x + \sum_{x=1}^r A_x \\ &= -(A_1 + A_2 + \dots + A_{l-1}) + (A_1 + A_2 + \dots + A_{l-1} + \dots + A_r) \\ &= -A_{l-1} - A_{l-2} \dots - A_1 + A_1 + A_2 + \dots + A_{l-1} + A_l + \dots + A_r \\ &(\because \text{補題 2 より}) \\ &= A_l + A_{l+1} + \dots + A_r \end{aligned}$$

これはちょうど A の区間 $[l, r]$ の和である. □

3.2 二次元累積和

二次元累積和は、一次元累積和を二次元配列に拡張したものである. 具体的には, $H \times W$ の二次元配列 A に対してクエリ (l_1, l_2, r_1, r_2) が与えられたとき

$$\sum_{x=l_1}^{r_1} \sum_{y=l_2}^{r_2} A_{x,y} \quad (3.3)$$

の値を高速に求めることができる.

3.2.1 定義

$(G, +)$ を可換群とする.

$H \times W$ の二次元配列 $A = \{A_{x,y}\}$ が与えられたとき, A の二次元累積和 $S = \{S_{i,j}\}$ は以下のよう定義される.

$$\begin{aligned} S_{i,0} &= 0, \\ S_{0,j} &= 0, \\ S_{i,j} &= \sum_{x=1}^i \sum_{y=1}^j A_{x,y} \quad (1 \leq i \leq H, 1 \leq j \leq W) \end{aligned} \quad (3.4)$$

3.2.2 構築

二次元累積和の構築は, 以下の Algorithm 1 を用いて $O(HW)$ で行うことができる.

Algorithm 1 二次元累積和の構築

```
1: for  $i = 1$  to  $H$  do
2:   for  $j = 1$  to  $W$  do
3:      $S_{i,j} \leftarrow A_{i,j}$ 
4:   end for
5: end for
6:
7: { 累積和の計算 }
8: for  $i = 1$  to  $H$  do
9:   for  $j = 1$  to  $W$  do
10:     $S_{i,j} \leftarrow S_{i,j} + S_{i-1,j}$ 
11:   end for
12: end for
13: for  $i = 1$  to  $H$  do
14:   for  $j = 1$  to  $W$  do
15:     $S_{i,j} \leftarrow S_{i,j} + S_{i,j-1}$ 
16:   end for
17: end for
```

3.2.3 クエリ処理

二次元累積和のクエリ処理は、以下の定理 6. により $O(1)$ で行うことができる。

定理 6. 二次元配列 A の区間 $[l_1, r_1] \times [l_2, r_2]$ の和は、 $S_{r_1, r_2} - S_{l_1-1, r_2} - S_{r_1, l_2-1} + S_{l_1-1, l_2-1}$ で求めることができる。

証明. 累積和 $S_{i,j}$ は、 A の左上 $(0,0)$ から (i,j) までの部分和である。すなわち、

$$S_{i,j} = \sum_{x=1}^i \sum_{y=1}^j A_{x,y} \quad (3.5)$$

したがって、累積和の差分を計算すると、

$$\begin{aligned} S_{r_1, r_2} - S_{l_1-1, r_2} &= \sum_{x=1}^{r_1} \sum_{y=1}^{r_2} A_{x,y} - \sum_{x=1}^{l_1-1} \sum_{y=1}^{r_2} A_{x,y} \\ &= \sum_{x=l_1}^{r_1} \sum_{y=1}^{r_2} A_{x,y} \end{aligned} \quad (3.6)$$

同様に,

$$\begin{aligned}
S_{r_1, l_2-1} - S_{l_1-1, l_2-1} &= \sum_{x=1}^{r_1} \sum_{y=1}^{l_2-1} A_{x,y} - \sum_{x=1}^{l_1-1} \sum_{y=1}^{l_2-1} A_{x,y} \\
&= \sum_{x=l_1}^{r_1} \sum_{y=1}^{l_2-1} A_{x,y}
\end{aligned} \tag{3.7}$$

以上より,

$$\begin{aligned}
(S_{r_1, r_2} - S_{l_1-1, r_2}) - (S_{r_1, l_2-1} - S_{l_1-1, l_2-1}) &= \sum_{x=l_1}^{r_1} \sum_{y=1}^{r_2} A_{x,y} - \sum_{x=l_1}^{r_1} \sum_{y=1}^{l_2-1} A_{x,y} \\
&= \sum_{x=l_1}^{r_1} \sum_{y=l_2}^{r_2} A_{x,y}
\end{aligned} \tag{3.8}$$

これはちょうど A の区間 $[l_1, r_1] \times [l_2, r_2]$ の和である.

□

第4章

提案手法

4.1 重み付き包除原理による考察

多次元に一般化する準備として、重み付き包除原理を用いて定理 6. を証明する.

証明. 二次元配列 A を $A_{i,j}$ が重みの重み付き集合であるとみなす. さらに A の部分集合 $W_{i,j}$ を以下のように定義する:

$$W_{i,j} = \{(x, y) \mid 1 \leq x \leq i, 1 \leq y \leq j\} \quad (4.1)$$

ここで,

$$\begin{aligned} \sum_{x=l_1}^{r_1} \sum_{y=l_2}^{r_2} A_{i,j} &= \sum_{i \in [l_1, r_1] \times [l_2, r_2]} A_{i,j} \\ &= \sum_{(i,j) \in W_{r_1, r_2}} A_{i,j} - \sum_{(i,j) \in |W_{l_1-1, r_2} \cup W_{r_1, l_2-1}|} A_{i,j} \\ &= \sum_{(i,j) \in W_{r_1, r_2}} A_{i,j} - \left(\sum_{(i,j) \in W_{l_1-1, r_2}} A_{i,j} + \sum_{(i,j) \in W_{r_1, l_2-1}} A_{i,j} - \sum_{(i,j) \in |W_{l_1-1, r_2} \cap W_{r_1, l_2-1}|} A_{i,j} \right) \\ &\quad (\because \text{重み付き包除原理より}) \end{aligned} \quad (4.2)$$

と表すことができ、特に $W_{i,j}$ の要素の重み総和が $S_{i,j}$ に等しいこと、および $|W_{l_1-1, r_2} \cap W_{r_1, l_2-1}| = W_{l_1-1, l_2-1}$ であることから、定理 4. より、

$$\sum_{x=l_1}^{r_1} \sum_{y=l_2}^{r_2} A_{i,j} = S_{r_1, r_2} - S_{l_1-1, r_2} - S_{r_1, l_2-1} + S_{l_1-1, l_2-1} \quad (4.3)$$

であることが示された. □

この式は次のように解釈できる.

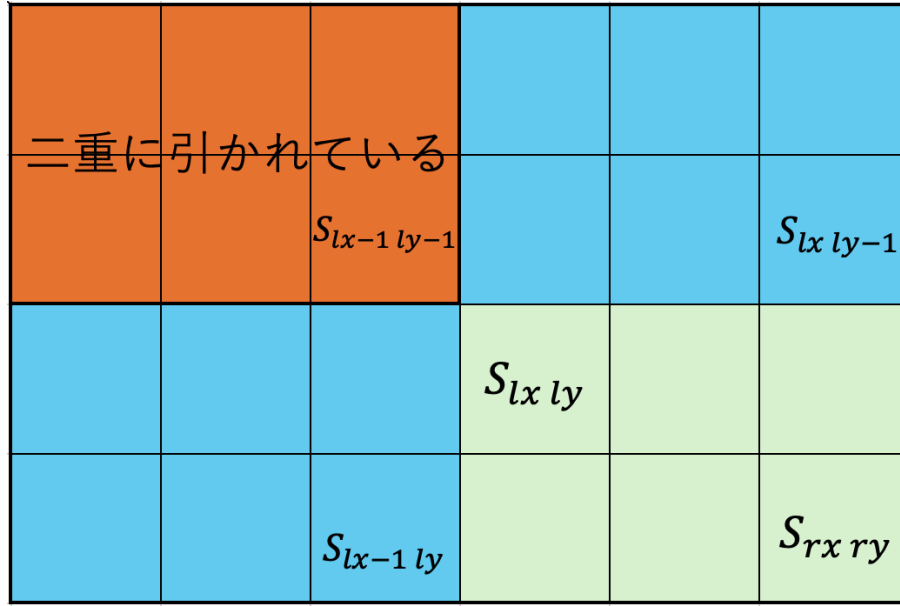


図 4.1 二次元累積和と重複補正の可視化

- S_{r_1, r_2} は $[1, r_1] \times [1, r_2]$ にあるすべての重みの合計である.
- S_{l_1-1, r_2} を引くことで, $[1, l_1 - 1] \times [1, r_2]$ の範囲を除去する.
- S_{r_1, l_2-1} を引くことで, $[1, r_1] \times [1, l_2 - 1]$ の範囲を除去する.
- しかし, $[1, l_1 - 1] \times [1, l_2 - 1]$ の範囲が二重に引かれているため, S_{l_1-1, l_2-1} を加える.

4.2 多次元累積和

4.2.1 定義

$(G, +)$ を可換群とする.

D 次元配列 $A = \{A_{x_1, x_2, \dots, x_D}\}$ が与えられたとき, A の多次元累積和 $S = \{S_{x_1, x_2, \dots, x_D}\}$ は以下のように定義される. ここで, $H_1, H_2, \dots, H_D$ をそれぞれ A の各次元の長さとする. 特に要素数を N とすれば, $\prod_{d=1}^D H_d = N$ である.

$$\begin{aligned}
 S_{x_1, 0, 0, \dots, 0} &= 0, S_{0, x_2, 0, \dots, 0} = 0, \dots, S_{0, 0, 0, \dots, x_D} = 0, \\
 S_{x_1, x_2, \dots, x_D} &= \sum_{x_1=1}^{x_1} \sum_{x_2=1}^{x_2} \dots \sum_{x_D=1}^{x_D} A_{x_1, x_2, \dots, x_D} \quad (1 \leq x_1 \leq H_1, 1 \leq x_2 \leq H_2, \dots, 1 \leq x_D \leq H_D)
 \end{aligned} \tag{4.4}$$

4.2.2 構築

多次元累積和の構築は, 以下の Algorithm 2 を用いて $O(\prod_{d=1}^D H_d) = O(N)$ で行うことができる.

Algorithm 2 D 次元累積和の構築

```
1: { 初期化 }
2: for  $x_1 = 1$  to  $H_1$  do
3:   for  $x_2 = 1$  to  $H_2$  do
4:     ...
5:     for  $x_D = 1$  to  $H_D$  do
6:        $S_{x_1, x_2, \dots, x_D} \leftarrow A_{x_1, x_2, \dots, x_D}$ 
7:     end for
8:   end for
9: end for
10:
11: { 累積和の計算 }
12: for  $x_1 = 1$  to  $H_1$  do
13:   for  $x_2 = 1$  to  $H_2$  do
14:     ...
15:     for  $x_D = 1$  to  $H_D$  do
16:        $S_{x_1, x_2, \dots, x_D} \leftarrow S_{x_1, x_2, \dots, x_D} + S_{x_1-1, x_2, \dots, x_D}$ 
17:     end for
18:   end for
19: end for
20: for  $x_1 = 1$  to  $H_1$  do
21:   for  $x_2 = 1$  to  $H_2$  do
22:     ...
23:     for  $x_D = 1$  to  $H_D$  do
24:        $S_{x_1, x_2, \dots, x_D} \leftarrow S_{x_1, x_2, \dots, x_D} + S_{x_1, x_2-1, \dots, x_D}$ 
25:     end for
26:   end for
27: end for
28: ...
29: for  $x_1 = 1$  to  $H_1$  do
30:   for  $x_2 = 1$  to  $H_2$  do
31:     ...
32:     for  $x_D = 1$  to  $H_D$  do
33:        $S_{x_1, x_2, \dots, x_D} \leftarrow S_{x_1, x_2, \dots, x_D} + S_{x_1, x_2, \dots, x_D-1}$ 
34:     end for
35:   end for
36: end for
```

4.2.3 クエリ処理

ここでは累積和の添字をビット表記で管理し、重み付き包除原理を用いて多次元累積和のクエリ処理アルゴリズムを導出する。

定義 4 (添字のビット表現). 添字 $y_1, y_2, \dots, y_D$ に対してビットマスク $\mathbf{y}$ を以下のように定義する.

$$\mathbf{y}_i = \begin{cases} 1 & (y_i = r_i) \\ 0 & (y_i = l_i - 1) \end{cases} \quad (4.5)$$

定義 5 (D 次元の添字集合). D 次元配列 A に対して, 集合 $W_{\mathbf{y}}$ を以下のように定義する.

$$W_{\mathbf{y}} = \{(x_1, x_2, \dots, x_D) \mid 1 \leq x_1 \leq y_1, 1 \leq x_2 \leq y_2, \dots, 1 \leq x_D \leq y_D\} \quad (4.6)$$

補題 3. K を正整数とする. K 個の集合 $W_{\mathbf{y}_1}, W_{\mathbf{y}_2}, \dots, W_{\mathbf{y}_K}$ に対して, 以下の式が成り立つ. ここで, $\&$ 記号はビットごとの論理積を表す.

$$\bigcap_{i=1}^K W_{\mathbf{y}_i} = W_{\mathbf{y}_1 \& \mathbf{y}_2 \& \dots \& \mathbf{y}_K} \quad (4.7)$$

証明. 左辺の $\bigcap_{i=1}^K W_{\mathbf{y}_i}$ は, すべての $1 \leq i \leq K$ に対して $(x_1, x_2, \dots, x_D) \in W_{\mathbf{y}_i}$ であるような添字の集合である. これは, すべての i について条件を満たす必要があるため, $\bigcap_{i=1}^K W_{\mathbf{y}_i}$ の d 番目の要素 x_d に対する上限は各 $\mathbf{y}_i$ の d 番目の要素 $y_{i,d}$ の最小値, すなわち $\min(y_{1,d}, y_{2,d}, \dots, y_{K,d})$ となる. いま, 各 i に対して $\mathbf{y}_i$ の d 番目のビットが 1 であるかどうかは $y_{i,d} = r_d$ であるかどうかと等価であり, 一般に $l_d - 1 < r_d$ であることから, $\bigcap_{i=1}^K W_{\mathbf{y}_i}$ の d 番目の要素 x_d は $\mathbf{y}_1 \& \mathbf{y}_2 \& \dots \& \mathbf{y}_K$ の d 番目のビットと等しい. $\square$

定理 7. D 次元配列 A の区間 $[l_1, r_1] \times [l_2, r_2] \times \dots \times [l_D, r_D]$ の和は, 以下の式で求めることができる. ここで, $\text{popcount}(x)$ は x を二進数表記した時の 1 の数を数える関数である.

$$\sum_{\mathbf{m}=0}^{2^D-1} (-1)^{\text{popcount}(\mathbf{m})-(D \bmod 2)} S_{\mathbf{m}} \quad (4.8)$$

証明. $\mathbf{r} = (r_1, r_2, \dots, r_D), \mathbf{l}_d - 1 = (r_1, r_2, \dots, l_d - 1, \dots, r_D)$ とすると, 区間 $[l_1, r_1] \times [l_2, r_2] \times \dots \times [l_D, r_D]$ は, D 次元の添字集合 $W_{\mathbf{r}}$ から $\bigcup_{d=1}^D W_{\mathbf{l}_d - 1}$ を引いたものである. したがって, 重

み付き包除原理より

$$\begin{aligned}
\sum_{x_1=l_1}^{r_1} \sum_{x_2=l_2}^{r_2} \cdots \sum_{x_D=l_D}^{r_D} A_{x_1, x_2, \dots, x_D} &= \sum_{(x_1, x_2, \dots, x_D) \in W_{\mathbf{r}}} A_{x_1, x_2, \dots, x_D} \\
&- \sum_{(x_1, x_2, \dots, x_D) \in \bigcup_{d=1}^D W_{1_d-1}} A_{x_1, x_2, \dots, x_D} \\
&= \sum_{(x_1, x_2, \dots, x_D) \in W_{\mathbf{r}}} A_{x_1, x_2, \dots, x_D} \\
&- \sum_{i_1 < i_2} \sum_{(x_1, x_2, \dots, x_D) \in W_{1_{i_1}-1} \cap W_{1_{i_2}-1}} A_{x_1, x_2, \dots, x_D} \\
&+ \sum_{i_1 < i_2 < i_3} \sum_{(x_1, x_2, \dots, x_D) \in W_{1_{i_1}-1} \cap W_{1_{i_2}-1} \cap W_{1_{i_3}-1}} A_{x_1, x_2, \dots, x_D} \\
&- \dots \\
&+ (-1)^D \sum_{(x_1, x_2, \dots, x_D) \in \bigcap_{d=1}^D W_{1_d-1}} A_{x_1, x_2, \dots, x_D}
\end{aligned} \tag{4.9}$$

が成り立つ。ここで、右辺第 k 項 ($1 \leq k \leq D+1$) は、 k 個の集合の積集合の重み和を表しており、補題 3 より添字をビット表記した際の 1 の個数がちょうど $D-k+1$ 個であるような集合についての和である。また右辺第 k 項の符号は $(-1)^{k+1}$ であり、 D が偶数のとき $(-1)^{\text{popcount}(\mathbf{m})}$ 、奇数のとき $(-1)^{\text{popcount}(\mathbf{m})-1}$ である。

よって求める和は、 D が偶数のとき

$$\sum_{k=1}^{D+1} (-1)^{k+1} \sum_{\{\mathbf{m} | 0 \leq \mathbf{m} \leq 2^D-1, \text{popcount}(\mathbf{m})=D-k+1\}} S_{\mathbf{m}}$$

D が奇数のとき

$$\sum_{k=1}^{D+1} (-1)^k \sum_{\{\mathbf{m} | 0 \leq \mathbf{m} \leq 2^D-1, \text{popcount}(\mathbf{m})=D-k+1\}} S_{\mathbf{m}}$$

であり、これをまとめると

$$\sum_{\mathbf{m}=0}^{2^D-1} (-1)^{\text{popcount}(\mathbf{m})-(D \bmod 2)} S_{\mathbf{m}} \tag{4.10}$$

が得られる。 □

以上より、区間 $[l_1, r_1] \times [l_2, r_2] \times \dots \times [l_D, r_D]$ の和を求めるクエリ処理は、以下の Algorithm 3 により $O(2^D)$ で行うことができる。 $\text{popcount}(x)$ は x を二進数表記した時の 1 の数を数える関数であり、計算量は $O(1)$ を仮定する。

Algorithm 3 多次元累積和のクエリ処理

```
1: { 区間の総和を求める }
2:  $sum \leftarrow 0$ 
3: for  $mask = 0$  to  $2^D - 1$  do
4:    $(y_1, y_2, \dots, y_D) \leftarrow (l_1 - 1, l_2 - 1, \dots, l_D - 1)$ 
5:   for  $i = 1$  to  $D$  do
6:     if  $(mask \text{ bitwise AND } (1 \ll (i - 1))) = 0$  then
7:        $y_i \leftarrow r_i$ 
8:     end if
9:   end for
10:  { 符号の決定 }
11:  if  $\text{popcount}(mask) \bmod 2 = D \bmod 2$  then
12:     $sum \leftarrow sum + S_{y_1, y_2, \dots, y_D}$ 
13:  else
14:     $sum \leftarrow sum - S_{y_1, y_2, \dots, y_D}$ 
15:  end if
16: end for
17: return  $sum$ 
```

第 5 章

今後の展望

本研究で提案する手法は特に高次元データを取り扱う場面で有効であり，データ解析や機械学習の分野における特徴量抽出の高速化などに応用が期待される．また，今後の展望として実際のプログラムとしての実装と速度検証が挙げられる．

謝辞

本研究をするにあたり，さまざまな面でご指導，ご意見を承りました一関工業高等専門学校未来創造工学科情報・ソフトウェア系千田栄幸教授，副査の小保方幸次教授，小林健一准教授に深く感謝いたします。また，共に研究に取り組んだ同研究室の齋暖斗氏，OpenAI ChatGPT，GitHub Copilot に深く感謝いたします。

参考文献

- [1] Guy E. Blelloch, "Prefix Sums and Their Applications", School of Computer Science, Carnegie Mellon University, CMU-CS-90-190, Nov. 1990.
- [2] 守屋悦朗, "コンピュータサイエンスのための離散数学", サイエンス社, 1992
- [3] Wikipedia, "Inclusion–exclusion principle", 2025, https://en.wikipedia.org/wiki/Inclusion%E2%80%93exclusion_principle (参照 2025-01-20)
- [4] 雪江明彦, "群論入門 第2版", 日本評論社, 2023,
- [5] Wikipedia, "Prefix sum", 2025, https://en.wikipedia.org/wiki/Prefix_sum (参照 2025-01-20)
- [6] 斎藤信男, 西原清一, "データ構造とアルゴリズム", コロナ社, 1998